Talking points

QIR Optimizer · BCSE307 Review 3 · Speaker 1: scenes 1–5 and 16 · Speaker 2: 6–9 · Speaker 3: 10–12 · Speaker 4: 13–15 and 17

SPEAKER 1

1. Title: A self-verifying optimizer for quantum programs

Core message: we built a compiler that makes quantum programs smaller, and proves every change is safe.

- **What the project is:** a compiler for quantum programs. Its main job is optimization: removing work the program doesn't need, without changing what it computes.
- **Three jobs, one per point on screen:**
  - **Compiles** quantum programs into QIR, the industry-standard intermediate representation, which is built on LLVM (the same infrastructure behind Clang and Rust).
  - **Removes** every gate that provably has no effect.
  - **Proves** each change. Before keeping a rewrite, a simulator checks that the program still behaves exactly the same.
- **The figure, which is the whole talk in miniature:** a small program with 5 gates and an if-statement. During replay:
  - each step names the rule used and shows a "validated" tag;
  - it ends with 2 gates, no branch, and "proven equivalent".
- **Hand-off line:** "The rest of the presentation explains how each of those steps works."

2. Quantum computers make mistakes

Core message: every gate costs accuracy, so removing gates directly improves results.

- **Hardware is noisy.** Typical error rates are about 0.2% per single-qubit gate and about 1.5% per two-qubit gate.
- **The errors compound.** The chance of an error-free run is all the individual success chances multiplied together:
  - with 20 gates, most runs are clean;
  - with 300 gates, almost none are.
- **Why a compiler helps:** fewer gates means a higher chance of a correct answer, on the same hardware. It is the cheapest accuracy improvement there is.
- **Concrete number:** a 30-gate adder from our benchmarks goes from a 39% to a 94% chance of an error-free run after optimization.
- **If asked where the numbers come from:** they are typical published error rates used in a simulated noise model, not measurements on real hardware. We say so everywhere it appears.

3. A qubit in one minute

Core message: only four ideas are needed, and the key one is whether a qubit's value is known.

- **Qubit:** like a bit, 0 or 1, but it can also be in a mixture of both, called superposition.

- **Measurement:** reading a qubit always gives a plain 0 or 1. The probabilities depend on the mixture, and reading it destroys the mixture.
- **In the figure:**
  - start at 00, and every run gives 00;
  - after H, q0 is 50/50, so the outcome is random;
  - after CX, only 00 or 11 ever appear, never 01 or 10.
- **Entanglement:** after that CX the qubits are linked. Neither has a value of its own; only the pair does.
- **The idea that drives our optimizer:** for every qubit, the compiler asks one question: is it definitely 0, definitely 1, or unknown? Entangled or superposed qubits are unknown.
- **If asked why the compiler can't just run the program to find out:** quantum states can't be copied, and looking at one changes it. Everything must come from analysing the program text, not from executing it.

4. Gates: the instructions

Core message: gates are instructions, and almost all of them undo themselves.

- **How to read a circuit:** each horizontal line is a qubit, time runs left to right, and each box or dot is a gate.
- **Single-qubit gates:**
  - **X** is NOT: 0 ↔ 1.
  - **H** turns a plain 0 or 1 into a 50/50 mix.
  - **Z, S, T** are phase gates. They have no visible effect on their own, but they change outcomes when combined with other gates. Algorithms such as Grover's search use them.
  - **Rotations (Rz, Rx, Ry)** turn a qubit by an angle. Two rotations on the same axis add up: Rz(a) then Rz(b) equals Rz(a + b).
- **Multi-qubit gates:**
  - **CX (CNOT)** flips the target (⊕) only when the control (●) is 1.
  - **CCX (Toffoli)** needs both controls to be 1, like AND.
  - **SWAP** exchanges two qubits.
- **Measure** reads the answer and cannot be undone.
- **The key fact:** look at the last line of each entry. Almost every gate is its own inverse: apply it twice and nothing happens. Most of our optimizations come from that one property.

5. Some gates undo each other

Core message: the simplest optimization deletes pairs that cancel, and it must be proven safe.

- **The program:** H twice, X twice and CX twice, which is three cancelling pairs.
- **After replay:** all gates are gone, the measurements give exactly the same results (always 00), and the figure confirms "proven equivalent".
- **Link to Review 2:** our Review 2 prototype did a version of this by matching adjacent lines of text.
- **What's new since then:**
  - it is done on a real compiler IR rather than text;
  - it is safe in tricky cases;

    - it goes far beyond adjacent pairs.
- **Hand-off:** "Next, how quantum programs actually look inside a compiler."

SPEAKER 2

6. QIR: quantum programs written as LLVM IR

Core message: QIR mixes quantum and classical code in one IR, and that is what we exploit.

- **What QIR is:** the Quantum Intermediate Representation, a standard from the QIR Alliance. Industry players such as Microsoft, Quantinuum and NVIDIA support it.
- **Why it matters:** it sits between many quantum languages and many machines, so one optimizer at this level helps all of them.
- **It is ordinary LLVM IR.** Quantum operations are just written in a particular way.
- **The four highlighted lines:**
    1. **A gate is a function call,** for example `call @__quantum__qis__h__body(...)`.
    2. **A measurement writes a result** into a result slot.
    3. **That result is read back as a normal one-bit value** (an `i1`), as classical code.
    4. **The program branches on it** with an ordinary LLVM `br`, an if-statement.
- **The point:** a QIR program is quantum and classical code in one control-flow graph. Classical compilers see only half of it, and quantum tools see only the other half.

7. Nobody was optimizing both halves

Core message: there is a gap between classical and quantum optimizers, and we fill it.

- **LLVM's optimizer:**
    - it sees the branches and classical code;
    - a gate is a call to an unknown external function that might do anything, so LLVM has to keep every gate.
- **Quantum circuit tools (Qiskit, tket):**
    - they understand gate algebra, so they know H·H does nothing;
    - they work on circuits, and support for branches and classical code is limited.
- **This project:** it understands gates, sees the branches and classical code, and proves every change. It is the only row in the table that is "yes" across the board.

8. What we built

Core message: a complete compiler pipeline, every stage our own code, running live.

- **The circuit at the top** is the small program from the title. Beside it, in plain words:
    - set q0 to 1 and copy it onto q1;
    - apply H twice to q2;
    - measure q1 and branch on the result.
- **Three components:**
    - **Front-end:** OpenQASM, a standard quantum language, goes to QIR through a lexer, a parser and semantic checks.
    - **Optimizer:** QIR becomes smaller QIR.

    - **Verifier:** a simulator proves the optimized program equals the original.
- **The stages in the tabs, left to right:**
    1. **Tokens:** the lexer splits the text into words, each with its line and column.
    2. **AST:** the parser builds the syntax tree.
    3. **Checked IR:** semantic analysis checks names, sizes and indices, works out constant angles, and expands user-defined gates.
    4. **QIR:** IR generation emits LLVM IR.
    5. **Wire DAG:** the optimizer reads QIR back and builds a dependence graph showing which operations touch the same qubit.
    6. **Optimized QIR:** the passes run until nothing more changes, and the circuit at the top shrinks from 5 gates to 2.
    7. **Output:** both programs produce identical measurement results.
- **Credibility point:** every stage is our code. LLVM's own verifier accepts every QIR file we produce, which is checked in our tests.
- **If asked why we didn't write a C++ LLVM pass as planned in Review 2:** we wanted to build every compiler phase ourselves, including the front-end, and to run the real engine live. LLVM is still used, as the independent checker of our output.

9. The front-end catches mistakes

Core message: it is a real compiler front-end, with real error handling.

- **The example has five deliberate mistakes:**
    1. a missing semicolon;
    2. an unknown gate name (`hadamard`);
    3. a gate using the same qubit twice (physically impossible: qubits can't be copied);
    4. a division by zero in an angle;
    5. an index out of range.
- **All five are reported in one run.** The compiler doesn't stop at the first error.
- **Each message gives** the exact line and column, a caret under the problem, and a suggestion such as "did you mean h?".
- **Fixing one live** makes the compiler rerun instantly.
- **If asked how it keeps going after an error:**
    - after a syntax error, it skips to the next semicolon and resumes;
    - a missing semicolon at a line end is reported and then assumed, so the next line is still checked.
- **Hand-off:** "Now the optimizer itself."

SPEAKER 3

10. Optimization 1: follow each qubit's wire

Core message: look along each qubit, not along the text, and let gates slide past ones that can't interfere.

- **Old approach:** two gates had to be on neighbouring lines of text. A gate on a different qubit in between would block the optimization for no reason.

- **New approach:** follow each qubit's wire, its own sequence of operations. Gates on unrelated qubits are ignored.
- **Sliding (commutation):** some gates can swap places without changing the result. The rule is that on every qubit they share, both gates either only change phase, or only flip. H and SWAP never move.
- **The rotation example:**
  - two rotations on q1 have a CNOT between them, but the CNOT only uses q1 as its control;
  - a Z-rotation can't interfere with a control, so the first rotation slides through;
  - the two rotations meet, and because their angles add to zero, both disappear.
- **Trust:** the sliding rule was checked by machine for every pair of gates in every position on three qubits, more than a thousand cases, with no exceptions.
- **If asked whether the rule is complete:** no, and deliberately so. It never allows an illegal swap, but it can miss a legal one. Missing an optimization is safe; a wrong swap is not.

11. ...and never when it is unsafe

Core message: knowing when not to optimize matters as much as optimizing.

- **How to read it:** dashed brackets mark gates the optimizer refused to touch, and dotted brackets mark what stopped it.
- **Left figure:**
  - two H gates on q0 look like a cancelling pair, but a measurement sits between them;
  - measuring changes the state, so removing them would change the result, and they stay;
  - on q2, an X sits between two H gates, and X doesn't commute with H, so they stay too.
- **Right figure:**
  - the function takes two qubit parameters, %q0 and %q1, which could be the same physical qubit (aliasing, as with pointers in C);
  - so the optimizer won't reorder gates across them.
- **Transparency:** every refusal is recorded with a reason: measurement, reset, unknown function call, possible alias, non-commuting gate, or superposition.
- **Principle:** missing an optimization is acceptable; breaking a program never is.

12. Optimization 2: knowing a qubit's value (the main new idea)

Core message: track what each qubit's value is, and use that knowledge to delete classical code that can never run.

- **The analysis:**
  - for every qubit, the compiler records definitely 0, definitely 1, or unknown;
  - it records the same for measurement results and ordinary variables;
  - a program starts with all qubits at 0.
- **The chain on screen:**
  1. q0 is set to 1, so it is definitely 1.
  2. The CX controlled by q0 therefore always fires. It is replaced by a plain X, which is cheaper.
  3. q1 is now definitely 1, so measuring q1 always gives 1.
  4. The if-statement checks for 0, so its condition is always false.
  5. That block can never run, so it is deleted, and the remaining blocks merge.

6. With no branches left, the program can run on simpler hardware: it fits QIR's basic profile, which needs no mid-circuit feedback.

- **Result:** 5 gates and 3 blocks become 2 gates and 1 block, proven equivalent.
- **The superposition version:**
  - q0 now starts as a 50/50 mix, so nothing is known about q1;
  - the measurement is random and the branch may run;
  - the optimizer still removes the H pair, but keeps the CX and the branch, and explains why (the condition depends on a qubit in superposition).
- **Why this is new:** quantum knowledge ("this qubit is 1") flows into classical code (the measurement constant, the comparison, the branch). Neither LLVM nor circuit tools make that crossing.
- **Compiler terms:** constant propagation (sparse conditional constant propagation), constant folding, branch folding, unreachable-code elimination and block merging, as in the Dragon Book, with qubit values as the constants.
- **If asked whether it is safe with entangled qubits:** yes. A qubit is only marked known if it holds that value on every run and isn't linked to anything unknown. Entangled qubits are always "unknown".
- **If asked about complexity:** each value can only move from "not reached" to "known" to "unknown", so the analysis finishes quickly. It is linear in program size times the number of tracked values.
- **Hand-off:** "So how do we know all of this is correct?"

SPEAKER 4

---

13. Every change is proven

Core message: we check every change rather than trusting the optimizer, and we show that the checker works.

- **Translation validation:**
  - after each individual rewrite, a simulator runs the old and new versions;
  - it compares every possible measurement outcome, the probability of each, and the final quantum state;
  - any difference means the change is rejected.
- **Why it is better than "trust us":** it catches bugs in any optimization, including ones added later. This is an established compiler-verification technique (Pnueli 1998, Necula 2000).
- **Proving the checker isn't a rubber stamp:**
  - we built a deliberately unsafe optimizer that matches gates by name, as a naive text matcher would;
  - in all three cases it looks reasonable but changes the output probabilities (shown in red), and the checker rejects it;
  - ours is accepted every time.
- **Live random testing:**
  - programs are generated randomly with built-in redundancy, measurements and if-statements;
  - each is optimized and proven equivalent, live in the browser;

    - result: zero counterexamples for ours, and the unsafe one caught hundreds of times on the same programs.
- **Also in our test suite:** the same kind of checks over thousands more programs, plus LLVM verification of the output.

14. How we are different

Core message: on the same program, other approaches stop at the gates and we go through the branches.

- **Same small program, three optimizers:**
    - **LLVM-style (no gate knowledge):** keeps all 5 gates and 3 blocks, because gate calls are opaque to it.
    - **Gate rules only (what circuit optimizers like Qiskit do):** removes the two H gates, giving 3 gates, but can't see that the branch is dead, so 3 blocks remain.
    - **This project:** also turns the CX into an X, proves the branch dead and deletes it. That leaves 2 gates and 1 block, proven equivalent.
- **Honest positioning:**
    - not new: cancelling gates is standard, and tracking qubit values has been done on circuits, in RPO (CGO 2021) and QCP (SAS 2023), plus a 2026 paper on branches;
    - new: doing it directly on QIR, the industry IR, carrying it through real branches and classical code, and proving every single change automatically.
- **If asked whether QCP is more powerful:** for entangled qubits, yes, it tracks more. Our contribution is the layer and the integration, not a more precise analysis. A QCP-style analysis could plug into our framework as future work.

15. Results

Core message: a large measured improvement, and everything is verified.

- **Headline:** across 24 benchmark programs, 48% of all gates removed, twice the Review 2 prototype's 24%.
- **Reading the chart, per program:**
    - light bar: the Review 2-style approach (adjacent pairs);
    - middle bar: following qubit wires;
    - dark bar: the full optimizer.
- **The adders tell the story:** they barely shrink with gate rules alone, but shrink about 83–84% once qubit values are tracked. That is the new part doing the work.
- **Verification numbers:**
    - LLVM's verifier accepts every optimized program, 24/24;
    - the simulator proves equivalence for every program small enough to check exhaustively, 21 of 24 (the other three have too many measurement branches to enumerate);
    - all 69 automated tests pass.
- **If asked why the QFT round-trip isn't fully removed:** cancelling it needs a more advanced technique (phase-polynomial reasoning), which is future work. We report the result honestly.

SPEAKER 1, LIVE DEMO

16. Try it: any program

Core message: it is real and general, not a set of prepared slides.

- **What's available:** type any program or add gates, switch each optimization on or off, replay every step, and see results on a simulated noisy machine.
- **Good live examples:**
  - **X on q0, then CX from q0 to q1:** the CX turns into an X, because q0 is known to be 1.
  - **H, measure, H on one qubit:** refused, because of the measurement.
  - **The naive optimizer:** tick it and it is rejected in red.
  - **Turning off hybrid constant propagation:** shows exactly what that pass contributes.
- **Invite the evaluator:** "Would you like to give us a program to try?" This is the evaluator-selected input the guideline asks for.

SPEAKER 4, CLOSE

17. Limits and next steps

Core message: we know the boundaries of our work and have a plan for each one.

- **Limits:**
  - **Proof by simulation** grows exponentially with qubits, so it stops at about 20 qubits; beyond that we report "not checked" rather than pretend.
  - **Entangled qubits** are treated as unknown, so a more precise analysis would find more.
  - **Noise results** come from simulation, not real hardware.
- **Next steps:**
  - smarter rotation merging (phase polynomials);
  - a richer qubit analysis (QCP-style);
  - a version that plugs into LLVM's own pass system;
  - runs on real quantum hardware.
- **Publication potential:** the combination of value tracking on QIR, classical branch folding and built-in validation is a credible short workshop paper.
- **Close:** "Thank you. We're happy to take questions."